

CTF Write-up

Challenge: **Picking Letters** | Category: **Reversing** | Difficulty: **Easy/Medium**

1. Initial Discovery

Upon running the binary `./lorem`, the program outputs a block of Lorem Ipsum text ending with a truncated flag:

```
SK-CERT{34sy...}
```

Using tools such as `strings` and `objdump`, we observe a massive amount of standard Latin text stored inside the `.rodata` section.

Near the end of this section, a suspicious format string appears:

```
%.12s...}
```

In C, `%.12s` instructs `printf` to display at most 12 characters of a string.

The developers intentionally assembled the full flag in memory but used the precision modifier to hide most of it during normal execution.

2. Deep Dive: Static Analysis

Disassembling the `main` function reveals that the flag is manually constructed on the stack.

Instead of storing the flag as a single static string, the binary performs repeated operations like:

```
movzbl 0x473(%rax), %eax
mov     %al, -0x34(%rbp)
```

- `0x473(%rax)` → Offset into the Lorem Ipsum text.
- `-0x34(%rbp)` → Location on the stack where the character is stored.

The binary is effectively “picking” letters one at a time from the Lorem Ipsum data and assembling the flag character-by-character.

Solving this entirely through static analysis would require manually tracking every offset — a tedious and error-prone process.

3. Dynamic Analysis – The Smarter Approach

Since the binary must assemble the complete flag before printing it, the easiest solution is to pause execution right before the `printf` call and inspect memory directly.

Step-by-Step Process

1. Open the binary in GDB:

```
gdb ./lorem
```

2. Disassemble the `main` function and locate the `printf@plt` call.

3. Set a breakpoint before the print instruction:

```
break *main+650
```

4. Run the program:

```
run
```

5. When execution stops, inspect the `RSI` register:

```
x/s $rsi
```

On x86-64 Linux systems:

- `RDI` contains the format string.
- `RSI` contains the actual string being printed.

By inspecting memory directly, we bypass the `%.12s` limitation and recover the entire flag.

SK-CERT{34sy_70_d3bug_wh3n_y0u_h4v3_wh0l3_c0d3}

4. Lessons Learned

- **Output ≠ Reality:** Programs may hide data in memory even if it is not fully displayed.
- **Format String Tricks:** Precision modifiers such as `.12s` can effectively conceal information.
- **Register Awareness:** During reversing challenges, registers like `RSI` often contain the valuable runtime data.